Programming Principles (PROG71990)

Assignment 1 (Fall 2026)

The due date is October 2, 2026, at 11:59 PM

This assignment will make sure that you

1) have basic proficiency in using the Visual Studio (or other) tool,
2) the ability to write basic programs, and
3) the understanding of how to package your programs for submission to eConestoga.

- To that end, in addition to submitting the Visual Studio solution that contains your Visual Studio projects, you will need to take screenshots of each of your programs at the close of their execution.
- Every submission you make this term will be the same format: a single zip file (**No other submissions will be graded**) containing the solution file and the set of project directories (each containing all the files required to build and run the program).
- The assignment will be graded out of 20.

Specifications

1. Write a program that uses one printf() call to print your first name and last name on one line, uses a second printf() call to print your first and last names on two separate lines, and uses a pair of printf() calls to print your first and last names on one line. The output should look like this (but using your name):

```
Sumeet Kaur          ← First print statement
Sumeet               ← Second print statement
Kaur                 ← Still the second print statement
Sumeet Kaur          ← Third and fourth print statements
```

2. Write a C program that calculates the third angle of a triangle when the other two angles are provided by the user. The program should prompt the user to enter the values of the first and second angles in degrees. Using the fact that the sum of the three interior angles of a triangle is 180 degrees, calculate the value of the third angle. The program should display the result in the following format:

```
Enter the first angle: 60
Enter the second angle: 45
The third angle is: 75 degrees
```

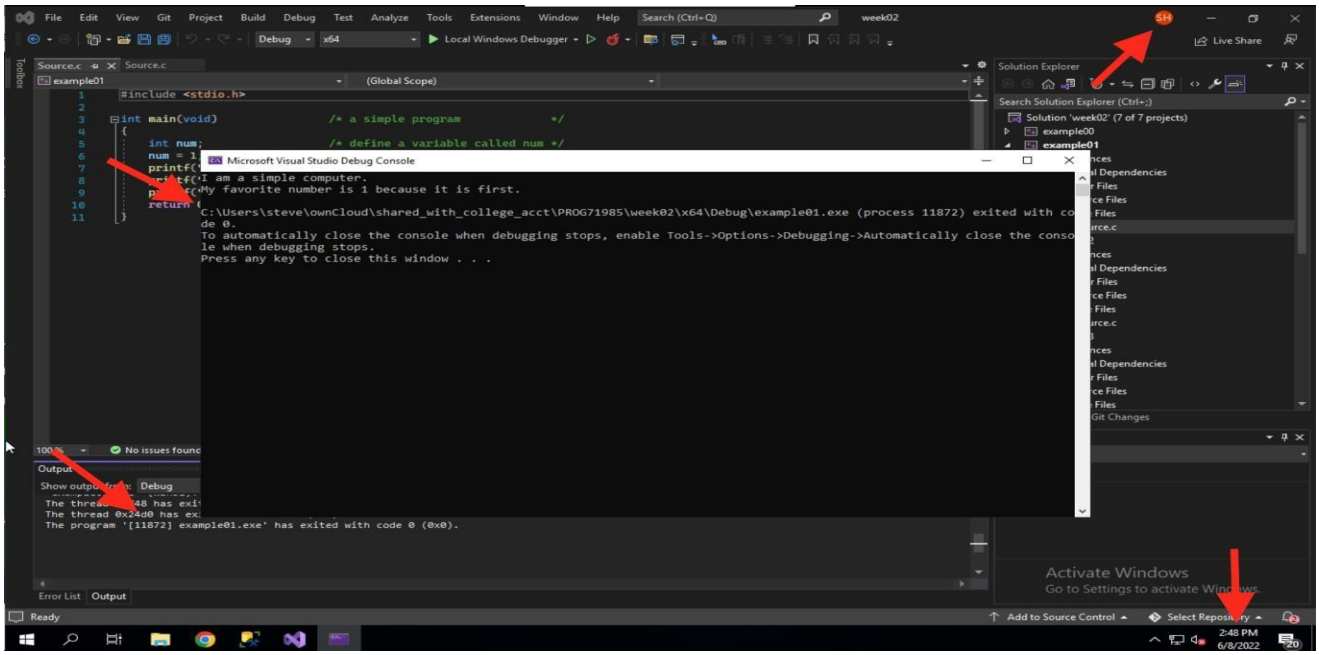
3. Write a C program that prompts the user to input two floats (allowing fractional numbers) and outputs their sum, product, and quotient. Make sure to include both sensible and informative prompts and output messages.

4. Write a C program that calculates an employee's total pay based on the number of hours worked and the employee's hourly wage. The program should prompt the user to enter the total number of hours worked and the hourly rate of pay. Both values should allow fractional numbers where appropriate. The total pay should be displayed as a value with two digits after the decimal point.

5. Write a C program that asks the user to enter the original price of an item and the discount percentage. The program should display the discount amount and the final price of the item after the discount is applied. Use clear prompts and informative output messages, and display monetary values with two digits after the decimal point.

Screenshot format

Your screenshot should look like the following: a screenshot showing the program having been executed, the build output, the date and time (lower right), and (optionally) your initials, as described by being signed into VS-online. I have illustrated the 4 important fields in the following screenshot:



Marking Rubric

Criteria	Level 4 (4 points)	Level 3 (3 points)	Level 2 (2 points)	Level 1 (0 points)
Specifications	The program works correctly and meets all stated specifications. Inputs, calculations, and outputs are handled correctly, and results are displayed in the required format.	The program works and produces the correct results. Output is displayed correctly, and most specifications are satisfied, with only minor omissions or inconsistencies.	The program produces generally correct results, but noticeable issues remain with formatting, prompts, output presentation, or other required specifications.	The program does not produce the required results, contains major functional errors, or fails to meet the main specifications.
Readability & Design	The code is exceptionally well organized and easy to read. Variable and function names are meaningful, formatting is consistent, and the	The code is generally well organized and readable. It demonstrates several good design practices, although	The code functions but has noticeable readability or design issues. Examples may include unclear variable names,	The code is poorly organized or difficult to read. Naming, formatting, or program structure

Criteria	Level 4 (4 points)	Level 3 (3 points)	Level 2 (2 points)	Level 1 (0 points)
	program demonstrates appropriate design practices taught in the course. Input and potential error conditions are handled appropriately where required. The code is easy to test, maintain, and extend.	some aspects of naming, structure, formatting, or input handling could be improved.	inconsistent formatting, unnecessary duplication, weak organization, or limited use of design practices discussed in the course.	significantly reduces readability, and the program would be difficult to test, maintain, or extend.
Reusability	The program is structured effectively using appropriate functions, modules, or ADTs where applicable. Components demonstrate single responsibility, low coupling, and high cohesion, making the code easy to reuse and extend.	The program demonstrates some appropriate decomposition into functions, modules, or ADTs. Most components have clear responsibilities, but there are still opportunities to improve reuse, coupling, or cohesion.	The program contains limited decomposition or reuse. Some repeated code or overly broad functions may be present, and responsibilities between components may not be clearly separated.	The program has little or no meaningful decomposition. Code is unnecessarily duplicated or tightly coupled, and responsibilities are not clearly separated.
Documentation	Documentation is clear, concise, and useful. Appropriate header documentation and comments explain the purpose of the program, important functions, parameters, assumptions, and non-obvious sections of the code without unnecessarily commenting obvious statements.	Documentation includes useful header information and appropriate comments. Most important sections of the program are explained, although some details may be missing or unclear.	Documentation is limited. Some comments or header information are provided, but they do not consistently help the reader understand the program's purpose, structure, or important parts.	Documentation is missing, very limited, inaccurate, or does not meaningfully help the reader understand the program.

Standard deductions

- 100% for submissions containing screenshots where the date displayed is not between the assignment 'available' date and the date stamp on eConestoga submission.
- 100% for any question that does not include requested/supporting screenshots
- 100% for any question whose requested/supporting screenshots are not of the full screen (taken from absolute outer edge to absolute outer edge (bezel to bezel)).
- 100% for any question whose screenshots do not have a date/time stamp.
- 100% for submissions 'packaged' (archived, compressed, etc.) using a tool other than the standard, built-in Windows compressed file tool.
- Regular late submission penalty (see Program Handbook)
- Penalties applied as per the Student Handbook for any plagiarism and/or academic dishonesty. Please note that the school considers submitting your own assignments from previous terms/classes to be an academic integrity incident.

****